

var关键字

2024年6月6日 星期四 23:34

类似于 JavaScript 中的var，它可以接收任何类型的变量，但最大的不同是 Dart 中 var 变量一旦赋值类型就会确定，则不能再改变其类型

```
1 var t = "hi world";  
2 // 下面代码在dart中会报错，因为变量t的类型已经确定为String，  
3 // 类型一旦确定后则不能再更改其类型。  
4 t = 1000;
```

Dart 也是一个强类型语言，在dart中在编译时会根据第一次变量赋值的类型来推断该变量的类型，编译之后其类型就被确定

Dynamic 和 Object

2024年6月6日 星期四 23:39

Object 是Dart所有对象的根基类，所有的类型都是Object的子类，包括Function和Null，所以任何类型都可以赋值给 Object 声明的对象。

Dybanuc 与 Object 声明的变量都可以赋值任意对象，且后期可以改变赋值的类型

```
1  dynamic t;
2  Object x;
3  t = "hi world";
4  x = 'Hello Object';
5  //下面代码没有问题
6  t = 1000;
7  x = 1000;
```

Dynamic 与 Object 不同的是 Dynamic 声明的对象编译器会提供所有可能的组合，而 Object 声明的对象只能使用 Object 的属性与方法，否则编译器会报错

```
1  dynamic a;
2  Object b = "";
3  main() {
4    a = "";
5    printLengths();
6  }
7
8  printLengths() {
9    // 正常
10   print(a.length);
11   // 报错 The getter 'length' is not defined for the class 'Object'
12   print(b.length);
13 }
```

但是这个特点使得在使用它的时候需要注意，容易引入一个运行时错误

```
1  print(a.xx); // a是字符串，没有"xx"属性，编译时不会报错，运行时会报错
```

Final 和 const

2024年6月6日 星期四 23:45

如果从未打算更改一个变量那么使用 final 或 const 不是一个 var 也不是一个类型，一个final 变量只能被设置一次

const 变量是一个编译时常量，只有在编译时直接替换为常量值

final变量在第一次使用时被初始化。

被final 或 const 修饰的变量，变量类型可以省略：

```
1 //可以省略String这个类型声明
2 final str = "hi world";
3 //final String str = "hi world";
4 const str1 = "hi world";
5 //const String str1 = "hi world";
```

空安全 (null-safety)

2024年6月6日 星期四 23:51

在dart中一切都是对象，这意味着如果定义了一个数字，在初始化之前使用了它，假如没有某种检查机制，则不会报错，比如：

```
1 test() {
2   int i;
3   print(i*8);
4 }
```

在有空安全之前上面的代码在执行前是不会报错的，但会出发一个运行时错误，原因是i的值为null。但现在有了空安全，则定义变量时我们可以指定变量时空还是不可空的。

```
1 int i = 8; //默认为不可空，必须在定义时初始化。
2 int? j; // 定义为可空类型，对于可空变量，我们在使用前必须判空。
3
4 // 如果我们预期变量不能为空，但在定义时不能确定其初始值，则可以加上late关键字，
5 // 表示会稍后初始化，但是在正式使用它之前必须得保证初始化过了，否则会报错
6 late int k;
7 k=9;
```

如果一个变量定义为可空类型，在某些情况下即使我们给它赋值过了，但是预处理器仍然有可能识别不出，这时我们就要显式（通过在变量后面加一个 “!” 符号）告诉与处理器它已经不是null了

```
1 class Test{
2   int? i;
3   Function? fun;
4   say(){
5     if(i!=null) {
6       print(i! * 8); //因为已经判过空，所以能走到这 i 必不为null，如果没有显式申明，则
7     }
8     if(fun!=null){
9       fun!(); // 同上
10    }
11  }
12 }
```

上面中如果函数变量为可空时，调用的时候可以用语法糖

1

```
fun?.call() // fun 不为空时则会被调用
```

函数声明

2024年6月7日 星期五 23:08

函数

2024年6月7日 星期五 23:02

Dart函数声明如果没有显式声明返回值类型时会默认当作 `dynamic` 处理，注意，函数返回值没有类型推断：

```
1 typedef bool CALLBACK();
2
3 //不指定返回类型，此时默认为dynamic，不是bool
4 isNoble(int atomicNumber) {
5     return _nobleGases[atomicNumber] != null;
6 }
7
8 void test(CALLBACK cb){
9     print(cb());
10 }
11 //报错，isNoble不是bool类型
12 test(isNoble);
```

对于只包含一个表达式的函数，可以使用简写语法：

```
1 bool isNoble (int atomicNumber)=> true ;
```

函数作为变量

2024年6月7日 星期五 23:09

```
1  var say = (str){  
2      print(str);  
3  };  
4  say("hi world");
```

函数作为参数传递

2024年6月7日 星期五 23:10

```
1 //定义函数execute, 它的参数类型为函数
2 void execute(var callback) {
3     callback(); //执行传入的函数
4 }
5 //调用execute, 将箭头函数作为参数传递
6 execute(() => print("xxx"))
```

可选的位置参数

1) 可选的位置参数

包装一组函数参数, 用[]标记为可选的位置参数, 并放在参数列表的最后面:

```
1 String say(String from, String msg, [String? device]) {
2     var result = '$from says $msg';
3     if (device != null) {
4         result = '$result with a $device';
5     }
6     return result;
7 }
```

下面是一个不带可选参数调用这个函数的例子:

```
1 say('Bob', 'Howdy'); //结果是: Bob says Howdy
```

下面是用第三个参数调用这个函数的例子:

```
1 say('Bob', 'Howdy', 'smoke signal'); //结果是: Bob says Howdy
```

可选的命名参数

定义函数时，使用{param1, param2, ...}，放在参数列表的最后面，用于指定命名参数。例如：

```
1 //设置[bold]和[hidden]标志
2 void enableFlags({bool bold, bool hidden}) {
3     // ...
4 }
```

调用函数时，可以使用指定命名参数。例如： `paramName: value`

```
1 enableFlags(bold: true, hidden: false);
```

可选命名参数在Flutter中使用非常多。注意，不能同时使用可选的位置参数和可选的命名参数。

可选命名参数在 Flutter 中使用非常多。注意，不能同时使用可选的位置参数和可选的命名参数

Mixin

2024年6月7日 星期五 23:15

Dart 不支持多继承，但是支持 mixin，简单来说 mixin 可以“组合”多个类

定义一个 Person 类，实现吃饭、说话、走路和写代码功能，同时定义一个 Dog 类，实现吃饭、和走路功能：

```
1  class Person {  
2      say() {  
3          print('say');  
4      }  
5  }  
6  
7  mixin Eat {  
8      eat() {  
9          print('eat');  
10     }  
11 }  
12  
13 mixin Walk {  
14     walk() {  
15         print('walk');  
16     }  
17 }  
18  
19 mixin Code {  
20     code() {  
21         print('key');  
22     }  
23 }  
24  
25 class Dog with Eat, Walk{}  
26 class Man extends Person with Eat, Walk, Code{}
```

我们定义了几个 mixin，然后通过 with 关键字将它们组合成不同的类。有一点需要注意：如果多个 mixin 中有同名方法，with 时，会默认使用最后面的 mixin 的，mixin 方法中可以通过 super 关键字调用之前 mixin 或类中的方法。我们这里只介绍 mixin 最基本的特性，关于 mixin 更详细的内容读者可以自行百度。

注意：如果多个 mixin 中有同名方法，with 时，会默认使用最后面的 mixin 的，mixin 方法中可以通过 super 关键字调用之前 mixin 或类中的方法。目前只学了 mixin 最基本的特性，更多可以深入学习

Future

2024年6月7日 星期五 23:28

Future 与 JS 中的 Promise 非常相似，同样是表示最终的成功或者失败，'.then' 和 '.catch'

由于本身功能很多，暂时只学习了常用的 API 及特性，请记住 Future 的所有 API 的返回值仍然是一个 Future 对象，所以可以很方便的进行链式调用。

Future.then

2024年6月7日 星期五 23:40

为了方便示例，在本例中我们使用 `Future.delayed` 创建了一个延时任务（实际场景会是一个真正的耗时任务，比如一次网络请求），即2秒后返回结果字符串"hi world!"，然后我们在 `then` 中接收异步结果并打印结果，代码如下：

```
1 Future.delayed(Duration(seconds: 2),(){
2   return "hi world!";
3 }).then((data){
4   print(data);
5 });
```

Future.catchError

2024年6月7日 星期五 23:41

2) Future.catchError

如果异步任务发生错误，我们可以在 `catchError` 中捕获错误，我们将上面示例改为：

```
1 Future.delayed(Duration(seconds: 2), () {
2     //return "hi world!";
3     throw AssertionError("Error");
4 }).then((data) {
5     //执行成功会走到这里
6     print("success");
7 }).catchError((e) {
8     //执行失败会走到这里
9     print(e);
10 });
```

在本示例中，我们在异步任务中抛出了一个异常，`then` 的回调函数将不会被执行，取而代之的是 `catchError` 回调函数将被调用；但是，并不是只有 `catchError` 回调才能捕获错误，`then` 方法还有一个可选参数 `onError`，我们也可以用它来捕获异常：

```
1 Future.delayed(Duration(seconds: 2), () {
2     //return "hi world!";
3     throw AssertionError("Error");
4 }).then((data) {
5     print("success");
6 }, onError: (e) {
7     print(e);
8 });
```

这里不同的是 `then` 方法还有一个可选参数 `onError` 可以用来捕获异常

Future.whenComplete

2024年6月7日 星期五 23:42

3) Future.whenComplete

有些时候，我们会遇到无论异步任务执行成功或失败都需要做一些事的场景，比如在网络请求前弹出加载对话框，在请求结束后关闭对话框。这种场景，有两种方法，第一种是分别在 `then` 或 `catch` 中关闭一下对话框，第二种就是使用 `Future` 的 `whenComplete` 回调，我们将上面示例改一下：

```
1 Future.delayed(Duration(seconds: 2),(){
2     //return "hi world!";
3     throw AssertionError("Error");
4 }).then((data){
5     //执行成功会走到这里
6     print(data);
7 }).catchError((e){
8     //执行失败会走到这里
9     print(e);
10 }).whenComplete(){
11     //无论成功或失败都会走到这里
12 };
```

Future.wait

2024年6月7日 星期五 23:42

有些场景中，我们需要等待多个异步任务都执行结束后才进行一些操作，比如有一个页面需要从两个网络接口获取数据，获取成功后，我们需要将两个接口进行特定的处理后再显示到UI界面上，这个时候就可以用 `Future.wait`，这个方法接受一个 `Future` 数组参数，只有数组中所有 `Future` 都执行成功后才会出发 `then` 的成功回调，只要有一个 `Future` 执行失败就会触发错误回调。

下面的例子通过模拟 `Future.delayed` 模拟两个数据获取的异步任务

```
1  Future.wait([
2    // 2秒后返回结果
3    Future.delayed(Duration(seconds: 2), () {
4      return "hello";
5    }),
6    // 4秒后返回结果
7    Future.delayed(Duration(seconds: 4), () {
8      return " world";
9    })
10  ]).then((results){
11    print(results[0]+results[1]);
12  }).catchError((e){
13    print(e);
14  });
```

执行上面代码，4秒后你会在控制台中看到“hello world”。

Async/await

2024年6月7日 星期五 23:47

不用看了，跟js的一样

Future中的回调地狱、消除回调地狱、使用async/await消除回调地狱 都跟js的一样

其实，无论是在 JavaScript 还是 Dart 中，`async/await` 都只是一个语法糖，编译器或解释器最终都会将其转化为一个 Promise（Future）的调用链。

Stream

2024年6月7日 星期五 23:49

Stream也是用于接受异步事件数据，和Future不同的是，它可以接受多个异步操作的结果（无论成功或失败）。也就是说，在执行异步任务时，可以通过多次触发成功或失败事件来传递结果数据或错误异常。Stream常用于会多次读取数据的异步任务场景，如网络内容下载、文件读写等。举个例子：

```
1 Stream.fromFutures([
2   // 1秒后返回结果
3   Future.delayed(Duration(seconds: 1), () {
4     return "hello 1";
5   }),
6   // 抛出一个异常
7   Future.delayed(Duration(seconds: 2), () {
8     throw AssertionError("Error");
9   }),
10  // 3秒后返回结果
11  Future.delayed(Duration(seconds: 3), () {
12    return "hello 3";
13  })
14]).listen((data){
15  print(data);
16}, onError: (e){
17  print(e.message);
18}, onDone: (){
19
20});
```

上面的代码依次会输出：

```
1 I/flutter (17666): hello 1
2 I/flutter (17666): Error
3 I/flutter (17666): hello 3
```

思考题：既然 Stream 可以接收多次事件，那能不能用 Stream 来实现一个订阅者模式的事件总线？

可以是可以，但是没懂为什么